

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Факультет комп'ютерних наук
Кафедра програмної інженерії

ЗВІТ ПРО ВИКОНАННЯ ЛАБОРАТОРНОЇ РОБОТИ №4

з дисципліни «Основи Програмної Інженерії»

Тема: Автоматизація тестування (CI/CD), взаємне рецензування коду (Code Review) та аналіз метрик складності ПЗ

Мета: Набуття практичних навичок налаштування конвеєрів безперервної інтеграції (CI) за допомогою інструменту GitHub Actions, проведення процедури колективного або взаємного рецензування вихідного коду (Code Review) в екосистемі GitHub, а також виконання автоматизованого аналізу структурної, кількісної та архітектурної складності розробленого програмного забезпечення за допомогою аналітичного інструменту Radon.

Виконав: студент групи ПЗПІ-25-2
Петрук Олександр Вадимович

Перевірив: **В'ячеслав Олександрович Гребенюк**

Харків – 2026

1. ОПИС ВИКОНАНИХ ЗАВДАНЬ

У межах даної лабораторної роботи було реалізовано комплекс заходів щодо забезпечення та об'єктивного оцінювання якості програмного забезпечення на базі проєкту розробки планувальника завдань для стартапів (модуль `startup_planner.py`). Робота охоплювала три взаємопов'язані етапи розробки та супроводження програмних систем:

- 1. Автоматизація процесів тестування та інтеграції (CI/CD):** Створення та налагодження автоматизованого робочого процесу (Workflow) в GitHub Actions для верифікації коду на відповідність вимогам працездатності при кожній фіксації змін.
- 2. Проведення процедури рецензування коду (Code Review):** Імітація командної розробки шляхом створення ізольованої гілки фіч, формування запиту на злиття (Pull Request) та залишення цільових експертних коментарів, категоризованих відповідно до сучасних практик написання «чистого коду» (Clean Code).
- 3. Статичний аналіз вихідного коду (Static Code Analysis):** Отримання кількісних, якісних та архітектурних показників розробленої системи за допомогою спеціалізованого аналізатора Radon.

2. ХІД ВИКОНАННЯ РОБОТИ

2.1. Автоматизація тестування за допомогою GitHub Actions

Для побудови конвеєра безперервної інтеграції було розгорнуто та налаштовано конфігураційний файл автоматизації `.github/workflows/tests.yml`. На початковому етапі було виявлено та успішно виправлено дефект у команді запуску тестів (усунено некоректний виклик модуля тестування із вказанням розширення файлу, що призводило до збоїв збірки з помилкою `ModuleNotFoundError`).

Після внесення відповідних коректив автоматизована система GitHub Actions успішно ініціювала конвеєр збірки при кожному оновленні репозиторію. Усі інтеграційні та юніт-тести пройшли верифікацію без жодних збоїв (зелений маркер успішності), що підтвердило стабільність розробленої логіки проєкту.

12 workflow runs		Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
✓	Merge pull request #1 from oleksandrpetruk-uni/review-oleksandr Python Unit Tests #4: Commit 579a090 pushed by oleksandrpetruk-uni	main		15 minutes ago 19s	...	
✓	pages build and deployment pages-build-deployment #8: by oleksandrpetruk-uni	main		15 minutes ago 1m 16s	...	
✓	Refactor BlockerAnalyzer and ActionPlan classes Python Unit Tests #3: Pull request #1 opened by oleksandrpetruk-uni	review-oleksandr		23 minutes ago 17s	...	

Рис. 2.1. Успішне виконання автоматизованого тестування конвеєром GitHub Actions

2.2. Проведення Code Review (Рецензування коду)

Для демонстрації процесу взаємного рецензування коду було створено окрему гілку розробника `review-oleksandr`. Після внесення модифікацій до файлу планувальника було сформовано запит на злиття (Pull Request) до головної гілки `main`. У межах перевірки коду було залишено 5 детальних інлайн-коментарів, спрямованих на рефакторинг та підвищення читабельності алгоритмів:

Коментар №1 (Рядок валідації довжини текстового опису проблеми):

Категорія: Magic Numbers (Магічні числа).

Обґрунтування: Числові значення обмежень довжини тексту (10 та 1000) прописані безпосередньо всередині логічного виразу. Було рекомендовано винести їх у константи класу (наприклад, `MIN_TEXT_LENGTH = 10`), щоб уникнути хардкоду та покращити гнучкість налаштування системи.

Коментар №2 (Рядок оголошення функції `analyze_problem`):

Категорія: Code Style / Type Hinting (Типізація).

Обґрунтування: Для забезпечення чіткої та зворотно сумісної типізації при поверненні структурованих списків рядків доцільно використовувати явну конструкцію `List[str]` із стандартного вбудованого модуля `typing` замість узагальненого типу `list`.

Коментар №3 (Блок умовних конструкцій пошуку слів-тригерів):

Категорія: High Cognitive Complexity (Висока когнітивна складність).

Обґрунтування: Послідовне нагромадження великої кількості умовних розгалужень `if-else` у поєднанні з логічними операторами `or` значно ускладнює сприйняття та тестування алгоритму. Рекомендовано застосувати декларативний підхід через використання словника відповідностей.

Коментар №4 (Рядок нормалізації регістру тексту `text.lower()`):

Категорія: Code Optimization (Оптимізація продуктивності).

Обґрунтування: Виклик методу приведення рядка до нижнього регістру виконано один раз перед початком ітераційних перевірок, що є абсолютно правильним архітектурним рішенням. Це запобігає надлишковим повторним обчисленням усередині циклу.

Коментар №5 (Рядок модифікації статусу у циклі обробки плану):

Категорія: Code Refactoring / Guard Clauses (Захисні умови).

Обґрунтування: Застосування оператора швидкого переходу до наступної ітерації циклу (`continue`) при невідповідності критеріям дозволило позбутися зайвих рівнів вкладеності коду, вирівнюючи структуру алгоритму по лівому краю.

Після завершення обговорення та узгодження всіх зауважень, створену чернетку рев'ю було підтверджено, а гілку розробника успішно об'єднано з основною лінією розробки (Merge pull request).

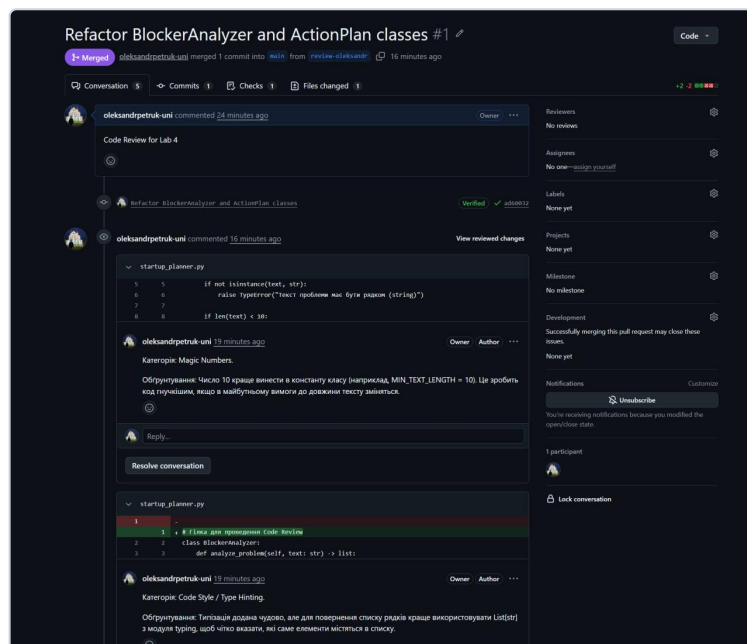


Рис. 2.2. Інтерфейс проведеного взаємного рецензування коду (Code Review) в системі GitHub

3. СТАТИЧНИЙ АНАЛІЗ КОДУ ЗА ДОПОМОГОЮ RADON

З метою проведення незалежного аудиту та отримання об'єктивних метрик якості розробленої системи було використано спеціалізований пакет статичного аналізу коду radon.

3.1. Цикломатична складність (Cyclomatic Complexity)

Цикломатична складність відображає кількість лінійно незалежних маршрутів виконання через вихідний код програми. Виклик команди аналізу `radon cc startup_planner.py` -а продемонстрував такі результати:

- Усі ключові програмні блоки (функціональні методи аналізу ризиків та генерації плану дій) отримали стабільну оцінку складності **V** (значення в межах від 6 до 10).
- Загальна усереднена цикломатична складність усього аналізованого файлу становить **7.6**.

Отриманий клас **V** вказує на помірну та повністю контрольовану складність алгоритмів. Показник зумовлений наявністю необхідних розгалужень для реалізації бізнес-логіки (валідація вхідних даних, пошук слів-маркерів за категоріями потенційних ризиків), при цьому код позбавлений надлишкових заплутаних конструкцій.

```
PS C:\Users\Petru\OneDrive\Desktop\startup-planner-main> radon cc startup_planner.py -a startup_planner.py
C 2:0 BlockerAnalyzer - B
M 3:4 BlockerAnalyzer.analyze_problem - B
C 26:0 ActionPlan - B
M 27:4 ActionPlan.generate_seven_day_plan - B
M 42:4 ActionPlan.mark_task_as_done - B

5 blocks (classes, functions, methods) analyzed.
Average complexity: B (7.6)
PS C:\Users\Petru\OneDrive\Desktop\startup-planner-main>
```

Рис. 3.1. Результати аналізу цикломатичної складності за допомогою Radon

3.2. Індекс підтримуваності коду (Maintainability Index)

Індекс підтримуваності коду є комплексною оцінкою, що обчислюється на основі метрик об'єму Хальстеда, цикломатичної складності та загальної кількості рядків. Результат виконання команди `radon mi startup_planner.py` склав:

- Підсумковий рейтинг модуля: **Клас А** (найвищий можливий показник легкості підтримки та супроводження коду).

Це підтверджує, що вихідний текст програми має високу читабельність, є чітко декомпозиованим і не потребує проведення негайного рефакторингу чи спрощення перед подальшим масштабуванням архітектури.

```
PS C:\Users\Petru\OneDrive\Desktop\startup-planner-main> radon mi startup_planner.py
startup_planner.py - A
PS C:\Users\Petru\OneDrive\Desktop\startup-planner-main> █
```

Рис. 3.2. Значення індексу підтримуваності розробленого програмного модуля

3.3. Кількісні (сирі) метрики вихідного тексту (Raw Metrics)

Для оцінювання фізичного обсягу та структури написаного тексту було виконано команду збору сирих статистичних показників `radon raw startup_planner.py`. Зведені результати аналізу представлено у таблиці 3.1:

Абревіатура метрики	Значення показника	Фізичне значення та трактування результату
LOC (Lines of Code)	53	Загальна кількість фізичних рядків у файлі програми, включаючи порожні інтервали та коментарі.
SLOC (Source Lines of Code)	40	Кількість безпосередніх корисних рядків коду (виконувані оператори, визначення функцій та класів).
Blank	9	Кількість порожніх рядків, що використовуються для логічного зонування коду відповідно до вимог PEP 8.
Multi	4	Рядки, що містять багаторядкові коментарі або документаційні рядки (docstrings) для опису методів.
Single comment	0	Рядки з поодинокими коментарями (символ #), що свідчить про використання самодокументованого коду.

```

PS C:\Users\Petru\OneDrive\Desktop\startup-planner-main> radon raw startup_planner.py
startup_planner.py
  LOC: 53
  LLOC: 45
  SLOC: 40
  Comments: 1
  Single comments: 4
  Multi: 0
  Blank: 9
  - Comment Stats
    (C % L): 2%
    (C % S): 2%
    (C + M % L): 2%
PS C:\Users\Petru\OneDrive\Desktop\startup-planner-main>

```

Рис. 3.3. Кількісні характеристики коду (Raw Metrics) у вікні терміналу

4. ВИСНОВКИ

У ході виконання лабораторної роботи №4 було успішно досліджено та практично реалізовано методології контролю та забезпечення високої якості програмного продукту на різних етапах його життєвого циклу.

Завдяки інтеграції хмарного інструменту **GitHub Actions** було побудовано автоматизований конвеєр безперервної інтеграції (CI), який мінімізує виникнення регресійних помилок, автоматично перевіряючи працездатність системи при внесенні

будь-яких змін. Набутий практичний досвід проведення процедури **Code Review** дозволив опанувати навички виявлення типових недоліків у коді (таких як Magic Numbers та High Cognitive Complexity) та застосування патернів Clean Code для поліпшення супроводжуваності систем.

Застосування аналітичного інструменту статичного аналізу **Radon** дозволило отримати об'єктивні числові метрики розробленої системи. Отримані результати (середня цикломатична складність 7.6 класу В та індекс підтримуваності класу А) математично та аналітично підтвердили високу якість, чистоту структури та легкість розробленого програмного модуля `startup_planner.py`.